

Quantum-Formulated Task Scheduling for Phone and PC Processing: A Speculative Study with Measured Classical Baselines

Harsh Goyal · Researcher · September 2026

Abstract

This paper asks whether quantum computing can improve the processing power of everyday devices, a laptop PC and an Android phone. It answers in two parts. The speculative part formulates workload scheduling, the assignment of tasks to CPU cores or to devices, as an Ising/QUBO problem and solves it with a simulated QAOA circuit. The empirical part implements the same scheduling problem in a working Python tool (**QSched**, in the QChip repository), executes the resulting schedules on real processes, and measures them on a 4-core/8-thread laptop. The results are modest and mixed. Replacing naive round-robin partitioning with a load-balancing scheduler gave measured speedups of $1.11\text{--}1.12\times$ (32 tasks) and $1.39\text{--}1.54\times$ (48 tasks). The QAOA solver found optimal or near-optimal splits for up to 12 tasks and beat classical longest-processing-time-first (LPT) there, but it never beat simulated annealing and fell behind LPT at 14 tasks. Measured makespans ran $1.5\text{--}2.0\times$ above predictions because of contention and power management. We conclude that today the gain comes entirely from the *scheduling formulation*, not from quantum hardware, and we state the conditions under which quantum solvers could matter.

1 Introduction

"Improving processing power" can mean raising peak throughput, which requires hardware, or using existing throughput better, which is a software problem. Quantum computers do not speed up general-purpose code on a phone or PC, and no near-term device will [1][2]. They may help with a narrower class of problems: combinatorial optimization, which includes scheduling [3][4].

Scheduling matters because static partitioning of work across cores is common (for example, splitting a job list evenly by count), and when task costs are heavy-tailed, the slowest core dictates total time. Offloading between a phone and a PC adds device speed differences and transfer cost. Finding the assignment that minimizes completion time (the *makespan*) is NP-hard [5].

Contributions:

1. An Ising formulation of two-machine scheduling and a state-vector QAOA solver (Section 3).
2. A working tool, QSched, with round-robin, LPT, annealing and QAOA schedulers, real task kernels, a multiprocess executor and a system monitor (Section 4).

3. Measured results on a laptop and a modeled phone–PC offload study, with the limits of each stated (Section 5).

This paper follows the QCMP study [6], which simulated a superconducting-qubit chip for audio analysis. Its conclusion also applies here: at this scale the simulation is a research vehicle, not a speedup.

2 Background

Makespan scheduling. Given tasks with costs c_i and machines with speeds v_j , choose an assignment minimizing $\max_j$ of (sum of c_i on j) / v_j . Graham's LPT rule sorts tasks by descending cost and places each on the machine that finishes it earliest; it is within $4/3$ of optimal on identical machines [5]. Simulated annealing [7] is a standard metaheuristic for the same problem.

Ising formulation. For two identical machines, let $z_i = \pm 1$ mark the machine of task i . The imbalance is $\sum c_i z_i$, and the makespan equals $(S + |\sum c_i z_i|)/2$ with $S = \sum c_i$. Minimizing the Ising energy $E(z) = (\sum c_i z_i)^2$ therefore gives the optimal schedule [4]. More machines need one-hot QUBO encodings, which cost $n \cdot m$ qubits.

QAOA. The Quantum Approximate Optimization Algorithm alternates a phase step $\exp(-i\gamma E)$ with a mixer $\exp(-i\beta \sum X)$ for p layers and tunes (γ, β) classically [3]. It is a candidate NISQ algorithm [2] with no proven advantage on this problem.

3 Method

Formulation. QSchd represents a schedule as an integer vector. For two machines it builds the diagonal of $E(z)$ over all 2^n bitstrings, simulates $p = 2$ QAOA layers on a state vector, tunes parameters by random search plus local refinement on $\langle E \rangle$, then evaluates the 32 most probable bitstrings classically and keeps the best. This is limited to $n \leq 18$ tasks by memory (2^n amplitudes).

Baselines and classical solvers.

- *Round-robin*: task i to machine $i \bmod m$ (the naive static split).
- *LPT*: Graham's rule, speed-aware.
- *Anneal*: simulated annealing over moves and swaps, seeded from LPT, with energy = makespan + $0.05 \cdot \sum(\text{times}^2)/\text{makespan}$. It is classical; "quantum-inspired" refers only to the energy-landscape framing.

Workloads. Four real CPU-bound kernels (matrix multiply, FFT round trips, prime sieve, SHA-256 chains) with sizes drawn log-uniformly, so a few tasks are far heavier than the rest. Each task is timed once on the host (best of two runs) to give its cost c_i .

Execution. One worker process per machine runs its assigned list sequentially. The reported *measured* time is the wall-clock time until all workers finish; the median of three runs is

reported.

4 Implementation

QSchd lives in `qcmp/sched/` and runs with only NumPy (psutil optional), so it also runs in Termux on Android.

Command	Purpose
<code>python -m qcmp.sched info</code>	CPU, RAM, battery, power plan snapshot
<code>python -m qcmp.sched bench</code>	Measured round-robin vs LPT vs anneal on real tasks; saves JSON to <code>results/</code>
<code>python -m qcmp.sched qaoa</code>	QAOA vs LPT vs anneal vs exact optimum on small instances
<code>python -m qcmp.sched offload</code>	Modeled phone–PC split with speed ratio and link cost

The test suite (`tests/test_sched.py`) checks that LPT beats round-robin on skewed costs, that annealing is within 2% of the exact optimum on 10 tasks, that QAOA reduces expected energy below the uniform state and lands within 15% of optimal, and that the scheduler respects device speeds.

5 Results

Test machine. Windows 11 laptop, AMD64, 4 physical / 8 logical CPUs, 7.8 GB RAM, "Balanced" power plan, running on battery (69%), with 81% RAM already in use and the CPU reporting about 1.0 GHz at the start of the run. These are unfavorable, noisy conditions, and we report them as measured.

5.1 Measured scheduling (4 workers)

Tasks	Scheduler	Predicted (s)	Measured (s)	Speedup vs round-robin
32	Round-robin	10.89	17.41	1.00×
32	LPT	9.12	15.65	1.11×
32	Anneal	9.12	15.59	1.12×
48	Round-robin	26.55	40.10	1.00×
48	LPT	14.59	28.88	1.39×
48	Anneal	14.59	25.98	1.54×

In the 32-task set a single 9.1 s task bounds every schedule, so there is little to gain; the predicted time drops 16% and the measured time drops 10%. In the 48-task set the schedulers have room to balance, and measured time drops 28% (LPT) to 35% (anneal). Annealing and LPT reach the same *predicted* makespan (both hit the lower bound), so the gap between them in measured time (25.98 s vs 28.88 s) is within what we would expect from run-to-run noise and contention,

not evidence that annealing is better. Each configuration was run three times and only medians are reported; we did not compute confidence intervals.

Prediction error. Measured times exceeded predictions by $1.5\text{--}2.0\times$. Likely causes are that 4 workers share 4 physical cores with other processes and hyper-threads, memory bandwidth contention among kernels timed in isolation, and thermal or power-plan limits on battery. We did not isolate which cause dominates. A practical consequence: schedules built from isolated timings are correct in *ordering* but over-optimistic in *magnitude*.

5.2 QAOA vs classical (2 machines, relative to exact optimum)

Tasks	LPT	QAOA (p = 2)	Anneal
6	1.000	1.000	1.000
8	1.007	1.000	1.000
10	1.021	1.002	1.000
12	1.015	1.001	1.000
14	1.005	1.031	1.000

QAOA matches or slightly beats LPT for 8–12 tasks, then falls behind at 14 tasks with the same optimizer budget. Annealing is optimal throughout. These are single random instances per size, so the comparison is indicative only. Also, QAOA here is a classical simulation whose cost doubles with every task; it cannot be faster than the classical methods it is compared with.

5.3 Modeled phone–PC offload

Using calibrated PC costs, an assumed phone speed of $0.35\times$ the PC, a 20 MB/s link and 2 MB transferred per phone task (**model assumptions, not measurements of a real phone**), 32 tasks complete in:

Plan	Modeled time (s)	Tasks on phone
All on PC	25.98	0
Round-robin	40.21	16
LPT (speed-aware)	20.85	16
Anneal	21.15	19

Naive alternating offload is 55% *slower* than doing everything on the PC, because half the work lands on the slower device. Speed-aware scheduling recovers a 20% gain over the PC alone. This shows why offload needs a scheduler; it does not establish real phone numbers. The same commands can be run in Termux to obtain them.

6 Discussion

Where the gain comes from. Every measured improvement in this study comes from classical load balancing. The quantum formulation contributed a clean problem statement, and QAOA produced valid but not superior schedules. This agrees with the general view that near-term quantum optimizers have not shown an advantage over strong classical heuristics [2].

When quantum could matter. Scheduling with large n , many machines, precedence and communication constraints produces search spaces where classical heuristics degrade. A fault-tolerant device with enough logical qubits could apply amplitude amplification for a quadratic speedup over exhaustive search [1], but that is not the regime where LPT and annealing already run in milliseconds. A scheduler that must decide in under a task's own runtime leaves no room for a cloud round trip to quantum hardware.

Beyond scheduling. Other ways to raise usable performance on these devices (thermal management, power plans, background-process control, memory pressure) are engineering measures. Our monitor exposes the relevant signals, and the laptop's low clock speed and high RAM use in Section 5 suggest they were limiting the run more than scheduling.

7 Limitations

- Only one machine was measured. No phone measurements are included; the phone results are modeled.
- One task set per size for benchmarks and one instance per size for QAOA; no statistical testing.
- Tasks are synthetic kernels, not real application workloads.
- Homogeneous cores only in the measured runs; heterogeneous big.LITTLE phone CPUs are not modeled.
- QAOA is simulated on a classical machine with a small parameter budget ($p = 2, 240$ evaluations); better tuning could change the 14-task result.
- Task costs come from isolated timing, which overstates parallel speed.

8 Future work

1. Run bench and offload inside Termux on the phone and add measured phone speed ratios and real link costs.
2. Add a real network executor so tasks actually run on both devices.
3. Repeat with multiple seeds and report confidence intervals; control background load and power plan.
4. Extend the formulation to precedence constraints and energy or thermal budgets, where classical baselines are weaker.
5. Test the QAOA path on cloud quantum hardware for small instances.

9 Conclusion

Quantum computing cannot raise the raw processing power of a phone or PC today. What it can contribute now is a formulation for scheduling, and the measurements here show that scheduling itself is worth doing: balanced schedules shortened measured completion time by 10–35% versus naive splitting, and speed-aware scheduling is essential for phone–PC offload. QAOA reproduced good schedules for small instances but did not outperform the best classical method. QSched provides a reproducible base for testing whether that changes at larger scale or on real hardware.

References

1. Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information*. Cambridge University Press.
2. Preskill, J. (2018). Quantum Computing in the NISQ Era and Beyond. *Quantum*, 2, 79.
3. Farhi, E., Goldstone, J., & Gutmann, S. (2014). A Quantum Approximate Optimization Algorithm. arXiv:1411.4028.
4. Lucas, A. (2014). Ising formulations of many NP problems. *Frontiers in Physics*, 2, 5.
5. Graham, R. L. (1969). Bounds on Multiprocessing Timing Anomalies. *SIAM Journal on Applied Mathematics*, 17(2), 416–429.
6. Goyal, H. (2025). A Quantum Computing Music Processor: Design and Simulation of a Superconducting Qubit-Based Chip for Audio Analysis.
7. Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671–680.